Boord - User & Admin Manual

Boord is a small, self-hosted app that tracks litchis from the moment they're picked in the field, through dispatch to the pack house, to receiving, wages, and reporting. It runs on one local server and is used from several devices at once - a phone or tablet at each field station, one or two devices at the pack house gate, and a computer in the farm office.

This manual is for everyone who touches the app:

- **Field capture staff** - picking teams who log crates as they're picked.
- **Pack house staff** - whoever receives incoming loads at the gate.
- **Farm admin / office staff** - whoever manages workers, wages, reports, and the server itself.

Table of Contents

1. Overview & Concepts

The three device roles

Every device that opens the app is one of exactly three roles:

Role	Used by	What it does
Field	Picking teams, one device per field station	Logs crates as they're picked, sends "picking slips" (loads) to the pack house
Pack House	Gate/receiving staff	Sees incoming loads, checks them in when the truck arrives
Admin	Farm office	Manages workers/teams/blocks/suppliers, calculates wages, runs reports, configures settings

A device only ever shows the screen for its own role - a field tablet never sees the admin screens, and vice versa. This keeps each device simple and hard to misuse by accident.

Why capture is offline-first

Field stations are often out of signal range. The field app is built so that **capturing a crate never depends on having a connection** - every crate is saved on the device first, and quietly synced to the server in the background whenever a connection is available. A picking team can keep working through a signal dropout without losing anything or needing to notice it happened.

Why the oldest-first, color-coded ordering matters

Litchis are highly perishable - quality drops fast once picked, and drops faster again once picked fruit sits waiting in the sun. Two places in the app use the same "traffic light" idea to make that visible at a glance:

- The **pack house's incoming queue** shows the oldest (longest-waiting) load first, colored **green → yellow → red** as it ages past the thresholds set in Settings (see chapter 11).
- The **admin Dashboard's Harvesting and In Transit lists** use the exact same coloring, so the office can see at a glance whether fruit is moving through fast enough.

This isn't just a UI nicety - it's the app's main quality-control signal: red means "this has been waiting too long, offload/process it before anything green."

---

2. Initial Server Setup

The app is a single Python service (FastAPI) that serves its own frontend and stores everything in a local SQLite database file - there's no separate database server, cloud account, or build step involved. It runs on one ordinary computer in the farm office (the "server"), and every other device (field phones/tablets, pack house devices) just opens a web address pointing at that computer - nothing needs to be installed on those other devices.

Prerequisites

- A Windows, Mac, or Linux computer that can be left switched on and connected to the network for the whole harvest season - this is the "server" referred to throughout this manual. It does not need to be powerful; any normal office PC is enough (see the recommended spec below).
- Python 3.9 or newer (3.11 is a safe default if installing fresh).
- Network access from every device that needs to reach the app (the same Wi-Fi/LAN the farm already uses, or a Tailscale network if devices need to reach it from outside the farm's own network - the app itself doesn't care which, it just needs to be reachable).
- The app's project folder, copied onto that computer (via USB drive, email/zip, or `git clone` - however it was given to you). This manual assumes it ends up at **`C:\Boord`** on Windows, or `~/Boord` on Mac/Linux; adjust the paths below if you use a different location.

Recommended server PC spec

This is a lightweight app (one Python process, a local SQLite database, no heavy computation) - it does not need server-grade hardware, just an ordinary PC that stays switched on.

Spec	Minimum	Recommended
OS	Windows 10/11 64-bit, macOS, or Linux	Same
CPU	Any dual-core from the last ~10 years (Celeron/ i3-class)	Any modern i3/Ryzen 3 or better
RAM	4 GB	8 GB
Storage	60 GB free	128 GB+ SSD
Network	Wi-Fi or Ethernet, same LAN as field/pack house devices	Ethernet, with a static IP or DHCP reservation

The app itself (Python plus all its dependencies) takes up around 150 MB. The database is small - a few hundred KB, growing slowly with the harvest records - and the 14 rolling backups are smaller

still, since they are only written on days something actually changed. Storage capacity is not a real constraint here.

A small **UPS (uninterruptible power supply)** is worth adding even though it's not strictly required: the one real risk on a farm is a power cut corrupting the SQLite database mid-write, and a UPS gives the PC enough time to either ride out a brief outage or shut down cleanly. No GPU or other special hardware is needed.

If the server is a **laptop** rather than a desktop, its own battery already covers this - a brief power cut just runs it on battery instead of risking a sudden shutdown mid-write, so a separate UPS isn't needed. See the lid-close note under Setting up on a Windows PC though - that's the one laptop-specific setting that still needs changing.

Getting the code onto the server: GitHub (recommended) or USB/zip

The app's source code is kept in a **private** GitHub repository at `github.com/DawiePieterse/Boord`. Deploying from GitHub is the recommended way to get it onto a new server, and also the way to pull future updates onto a server that's already running - it avoids re-copying files by hand and keeps a clear record of exactly what code is running. Copying the folder via USB drive or a zip file (see Prerequisites) still works fine if preferred; skip to Step 2 below if so.

Step A - Install Git for Windows. Go to `git-scm.com/download/win` and download/run the 64-bit installer. The default options on every screen are fine - just click "Next" through to "Install". This gives the PC the `git` command used below (Windows already includes the underlying `ssh` / `ssh-keygen` tools it needs, via its built-in OpenSSH client).

Step B - Generate an SSH key for this server. Since the repo is private, this PC needs its own key to prove it's allowed to read it. Open Command Prompt and run:

```
ssh-keygen -t ed25519 -C "farm-server"
```

Press Enter three times to accept the default file location and no passphrase. Then display the public key so it can be copied:

```
type %USERPROFILE%\.ssh\id_ed25519.pub
```

Step C - Add the key to GitHub as a deploy key.

1. In a browser, go to the repository on GitHub, then **Settings** → **Deploy keys** → **Add deploy key**.
2. Give it a title (e.g. "Farm server"), paste in the public key from Step B, and **leave "Allow write access" unchecked** - this server only needs to *read* the code, never push changes back, so a read-only key is the safer choice.
3. Click **Add key**.

Step D - Confirm the connection, then clone. Test the key against the repository specifically (a plain `ssh -T git@github.com` does **not** reliably confirm a deploy key - test against the actual repository URL instead):

```
git ls-remote git@github.com:DawiePieterse/Boord.git
```

This should print a list of branches/commits with no error. If it prints `Permission denied (publickey)`, the key from Step B wasn't added correctly in Step C - double check it and retry. Once it works, clone the repo to wherever the app should live, e.g.:

```
git clone git@github.com:DawiePieterse/Boord.git C:\Boord
```

Continue from Step 2 (Install Python) below - the clone replaces Step 1 (copying the folder by hand).

A fresh clone lands on the `main` branch, which is not a release. Once the server is otherwise set up, do the release-key steps below and then run `update_server.bat` once - that is what moves it onto the newest signed release and is where it stays from then on. The clone itself is the one moment you are trusting the network rather than a signature, so do it over a connection you control, not on someone else's WiFi.

Cutting a new release (developer). Farm servers install signed tags, so pushing to `main` deploys nothing on its own.

The whole release in one command, runnable from any directory:

```
~/Documents/Boord/scripts/ship.sh v2.14
```

It pushes `main`, offers the `frontend/shared/api.js` version bump if it is still on the old number, runs the signing helper below, and asks once more before pushing the tag - that last pause is where you check the fingerprint.

To do the signing step on its own, or to see exactly what it refuses and why, use the helper directly rather than tagging by hand:

```
scripts/release.sh v2.1
```

It refuses to proceed unless the working tree is clean, the tag looks like `v<number>`, that tag doesn't already exist, a `user.signingkey` is configured, and `Boord.VERSION` in `frontend/shared/api.js` matches the tag. That last check matters more than it looks: `Boord.VERSION` is what every screen prints in its header, and it is how you tell at a glance whether a device actually picked up an update - a header that disagrees with the deployed release is worse than no version display at all.

The tag is signed and verified locally, but **not pushed**. Push it yourself when you're ready for farms to see it:

```
git push origin v2.1
```

Never re-point a tag that has already been pushed. Servers compare the tag name they are on against the newest tag, so a moved tag silently fails to re-deploy on any server already sitting on it. Cut a new version instead.

Trusting the release key (one-time, per server). A Boord release is not "whatever is on the `main` branch" - it is a **tag signed with the release key**. `update_server.bat` refuses to install a tag that isn't signed by that specific key, which means simply being able to push to GitHub is not enough to ship code to a farm. That matters because the server runs as SYSTEM: without this, a stolen GitHub token would mean code execution on every farm running Boord.

For that check to work, the server needs to know two things - the public key itself, and which fingerprint to insist on. Only the second is manual.

`install.bat` does most of this for you. It installs Gpg4win if GnuPG is missing, points git at it, and imports the release key that ships in the repository as `release-key.asc`. What it cannot do is decide **which** key to trust - that is the one step a person has to perform.

The only manual step: record the fingerprint. In the project folder:

```
echo 67C64CFDD584DD140E58AF6E329C9B9DD0562A9D> data\release_key.fpr
```

Substitute the fingerprint you were given by whoever maintains this install. **There is no space before the `>`** - `echo foo > file` writes a trailing space into the file and the fingerprint will not match. Check it:

```
type data\release_key.fpr
```

40 characters, nothing else.

> **Why the key can ship in the repository but the fingerprint cannot.** They do different jobs. The key is just a key; the fingerprint is the decision about which key counts. An attacker who could push to the repository could swap `release-key.asc` for their own key and sign a release with it - and `data\release_key.fpr` is what catches that, because their fingerprint would not match the one on file. It lives in `data\`, which is gitignored, so no update can rewrite it. Treat that file as the thing that actually protects the farm, and change it only when you deliberately rotate the key.

If `data\release_key.fpr` is missing, `update_server.bat` stops and updates nothing. That is intentional: a server that cannot tell a genuine release from a tampered one should not be installing either. It also checks that GnuPG actually runs before trusting any signature check, so a missing gpg reports itself as a missing gpg rather than as a failed signature.

Doing it by hand. If you are not using `install.bat`, install Gpg4win from `gpg4win.org` (Git for Windows bundles a `gpg.exe` that does **not** work here - it keeps keys in a `keyboxd` daemon the Git distribution does not ship, so importing fails with `probably not installed` and processes zero keys). Then:

```
where gpg
git config --global gpg.program "C:/Program Files/GnuPG/bin/gpg.exe"
gpg --import release-key.asc
```

Open a new Command Prompt after installing, since PATH changes do not reach an already-open window.

Pulling future updates. Once a new signed release is tagged and pushed, update an already-running server.

The easy way: double-click **`update_server.bat`** at the top of the project folder. It fetches the release tags, picks the newest `v*` by version order, **verifies its signature against `data\release_key.fpr`**, checks it out, installs any new dependencies, and restarts the server (via the Scheduled Task) all in one step - this is the recommended way to deploy an update, since it's easy to forget the restart step if done by hand (a checkout alone does not restart anything, so the running server keeps serving the old code until it's explicitly restarted).

If the signature check fails, the script stops and changes nothing - the server keeps running the release it was already on. Do not "fix" this by checking the tag out by hand; find out why it failed first. The usual innocent cause is a rotated release key that this server wasn't told about.

It also **stops the server properly before touching the database**, which is more than ending the Scheduled Task: ending the task kills the launcher, and the server process it started can outlive it and carry on serving. The script checks port 8000 is genuinely free and refuses to migrate if it is not, because a schema change applied underneath a running server does not report an error - it just leaves the farm with a database two things disagree about.

Being told when a release is out. Nothing checks by itself unless you ask it to. Double-click **`setup_update_check.bat`** once and a Scheduled Task ("Boord Update Check") runs `update_server.bat --check` every morning at 07:30. That looks for a newer signed release, verifies its signature, and records what it found - **it installs nothing**. When there is one, Settings → Server in the admin app says so, and installing it stays a double-click of `update_server.bat` at a time that suits the pack house.

That task deliberately runs as **you**, not as SYSTEM, unlike the server and the heartbeat. Fetching from GitHub uses the deploy key in your own `%USERPROFILE%\.ssh`, and SYSTEM has a different profile with no key in it - a check running as SYSTEM would fail `Permission denied (publickey)` every morning, silently, forever. The trade is that the check only runs while you are logged on, which is also the only time anyone could act on what it finds. You can run it by hand at any time:

```
update_server.bat --check
```

The manual way, if you'd rather do each step yourself: open Command Prompt in `C:\Boord` and run:

```
git fetch --tags --force origin
git tag --list "v*" --sort=-v:refname
```

```
git verify-tag <newest-tag>
git checkout --force <newest-tag>
```

Check the `verify-tag` output names your release key **before** the checkout - that is the whole point of the exercise. The checkout leaves git in "detached HEAD" state, which is normal and correct here: the server tracks releases, not a branch.

Doing it by hand, remember the database step as well, with the server stopped:

```
backend\.venv\Scripts\python.exe backend\migrate.py
```

The checkout itself only touches the app's code - the database, worker photos and backups all live in the gitignored `data/` folder, and no checkout ever writes there. If `backend/requirements.txt` changed, re-run the installer (`install.bat`) or `pip install -r requirements.txt` to pick up any new dependencies. Since Boord started using proper schema migrations, that is not optional any more: a release that changes the database cannot run without them, so `update_server.bat` now stops rather than starting a server it knows will fail.

Then bring the database itself up to date - see Database changes on update below.

Then restart the server (see Stopping, starting, and restarting the server below).

Database changes on update

Some releases change the shape of the database - a new column, a renamed one, a table that did not exist before. The server applies those changes itself the first time it starts on the new release, so an ordinary update needs nothing from you.

`update_server.bat` does it a moment earlier, on purpose: with the server stopped, in its own window, while you are still standing there. A database change is the one part of an update that touches the farm's own records rather than just the code, and it should happen where somebody can see it go wrong - not inside a Scheduled Task at seven in the morning.

Nothing is changed until a copy has been made. Immediately before any database change, Boord writes a complete copy of the database to `data\backups\pre_migration_<date>_<time>_<what>.db` and keeps the three most recent. If that copy cannot be written - a full disk, almost always - the update stops and the database is left exactly as it was, which means the release you were already running still works.

These copies are deliberately different from the nightly backups next to them: they are plain `.db` files rather than zips, so putting one back is a single file copy. They are roughly the size of the database itself, which is a few hundred KB.

To go back to one, stop the server, copy the file over `data\boord.db` , then check out the release you were on before and start the server again. The pre-migration copies are never deleted by the nightly 14-backup pruner; only a fourth database change removes the oldest of them.

If a release changes the database and something goes wrong, the update stops with **THE DATABASE COULD NOT BE MIGRATED** and tells you the two commands to get back to the previous release. Send the message on rather than trying the update again.

Checking an install is sound. After a fresh install, a database restore, or any update that worries you, run the self-test:

```
backend\.venv\Scripts\python.exe scripts\selftest.py
```

It checks the parts of the app where a wrong answer would look plausible - the schema migration paths, the copy taken before a migration, the guards on bulk imports, and the setup wizard's view of how far this farm has got - along with the database's own schema: that this server is on the newest schema version, and that it matches what this release expects. It prints a line per check, ending in PASSED or FAILED. It reads only; it never writes to the database and needs no internet, so it is safe to run on the live server at any time, including while people are using the app. If anything reports FAILED, the message names what disagreed - send that text on rather than acting on the app's figures.

Either way, remember that each phone/tablet's installed app also needs a full close-and-reopen afterward to pick up the update - see Confirming devices picked up an update.

Removing Boord from a PC

Double-click **`uninstall.bat`**. It stops and unregisters the auto-start Scheduled Task, removes the heartbeat task and the firewall rule, deletes the generated `start_server.bat`, and removes the Python virtual environment. It then prints what is still on the machine.

It does not delete `data\`, and it does not delete the project folder. That folder holds the harvest database, worker photos and ID numbers, the wage history, every backup, and the release-key fingerprint - a season of records that reinstalling does not bring back. Deleting it stays a deliberate act, so the uninstaller prints the commands rather than running them:

```
xcopy /E /I "C:\Boord\data" "%USERPROFILE%\Desktop\boord-data-backup"
rmdir /s /q "C:\Boord"
```

Copy first, and run the second one from a folder outside `C:\Boord` - a Command Prompt sitting inside it will block its own delete.

Python, Git, Gpg4win, your GnuPG keyring and git's `gpg.program` setting are all left alone: other things on the PC may depend on them.

> **If a folder refuses to delete**, something still has a file open in it. Open Resource Monitor (`resmon`), go to the CPU tab, expand **Associated Handles** and search for the folder name - it will name the process. Cloud sync clients are the usual culprit, which is one more reason not to run the app from a synced folder.

Stopping, starting, and restarting the server (Task Scheduler)

If the server was set up via `install.bat` (or manual Step 12), it runs as a Scheduled Task named **"Boord Server"** that starts automatically at boot, as SYSTEM, with **no visible window** - there's no Command Prompt to close, so stopping/starting it goes through Task Scheduler instead.

Using the Task Scheduler app:

1. Open **Task Scheduler** (search for it in the Start menu).
2. Find **"Boord Server"** in the Task Scheduler Library.
3. Right-click it → **End** to stop the server, or **Run** to start it. To restart (e.g. after an update), do **End** then **Run**.

Using Command Prompt (must be "Run as administrator"):

```
schtasks /end /tn "Boord Server"
schtasks /run /tn "Boord Server"
```

Simplest restart: since the task launches automatically on every startup anyway, just restarting the PC has the same effect as End + Run.

If you need it definitely stopped - before copying the database, or restoring a backup - use **`stop_server.ps1`** rather than End on its own:

```
powershell -NoProfile -ExecutionPolicy Bypass -File stop_server.ps1
```

End stops the launcher, and the server process it started can survive and keep the database open. This script ends the task, then waits until port 8000 is actually free, and tells you what is holding it if it never comes free. It only ever stops processes running from Boord's own `backend\.venv` - if something else on the PC has taken port 8000, it says so and stops rather than killing it. `update_server.bat` calls it for you.

Checking it's actually running: browse to `http://localhost:8000/` on the server PC itself - if the device setup screen loads, it's up. There's no window to glance at for this, since the task runs headless.

Quick setup: the automated installer (recommended)

The project folder includes `install.bat`, which automates everything in the manual step-by-step section below - installing Python if it's missing, creating the virtual environment, installing dependencies, opening the firewall port, and registering the server to auto-start with Windows (no login or password needed for it to start). It's safe to double-click again later if something needs redoing - each step checks what's already in place first.

1. Get the project folder onto the PC, e.g. by cloning it from GitHub or copying it via USB/zip (see Step 1 below).
2. Double-click **`install.bat`** at the top of that folder.

3. If Windows shows a blue **"Windows protected your PC"** screen, click **"More info"**, then **"Run anyway"**. This is normal for any script that isn't from a large, registered publisher - it doesn't mean anything is wrong with it.
4. If a **User Account Control** prompt appears asking to let the app make changes, click **"Yes"** - administrator rights are needed to configure the firewall and register the auto-start task.
5. Wait for it to finish. A black window will print its progress (this can take a few minutes the first time, mostly spent downloading Python and the app's dependencies) and finish with the address to browse to from other devices. Press Enter to close the window when it says "Setup complete!".
6. **Note where the Admin app opens from.** There is no sign-in and no password. Boord has one admin, so what decides who reaches the Admin screens is which network the request comes from: **this PC, or a device on your Tailscale tailnet**. On this PC, browse to `http://localhost:8000/` and Admin opens straight up. From anywhere else you need Tailscale - see Connecting via Tailscale HTTPS.

The Field and Pack House screens are unchanged: any phone or tablet on the farm Wi-Fi still reaches them at the LAN address the installer printed. Only `/admin/` is restricted, and opening it from the farm Wi-Fi answers "*The Admin app is only reachable from the server itself or over Tailscale*". That is what keeps worker ID numbers, bank details and the payroll off the devices in the orchard now that there is no password in front of them.

If the installer fails partway, or you'd rather understand/do each part by hand, use the manual steps below instead - they're exactly what the installer automates.

Setting up on a Windows PC (step by step)

This is the most common setup, since most farm offices run Windows. Every step is done once, when first setting up the server. **Most farms can skip straight to the automated installer above** - use these steps instead if you prefer to do it by hand, or need to troubleshoot one specific part.

Step 1 - Get the app folder onto the PC. Either clone it from GitHub (recommended), or copy the whole project folder over via USB drive or a zip file. Either way, place it somewhere permanent and easy to find, e.g. `C:\Boord`. Avoid Desktop or Downloads, since those are more likely to get tidied up or deleted by accident.

> **Never put the app folder inside a synced cloud drive** - Google Drive, OneDrive, Dropbox or iCloud. Two things go wrong. The sync client keeps file handles open, which blocks updates and folder deletion (this is a real one: it took a reboot to work out why a folder refused to delete). More seriously, `data\` holds a live SQLite database plus every worker's ID number, bank details and photograph - syncing a database file while it is being written is a known way to corrupt it, and uploading worker records to a personal cloud account is a data-protection problem you do not want. Back up *to* a cloud drive by all means (chapter 14) - a backup zip is a finished file, which is exactly what sync is good at. Just do not run the app from one.

> **Set the farm's GPS location.** The weather shown in the header, and the conditions stamped onto every crate and picking note, are looked up for those coordinates. Until they are set the app fetches no weather at all - the header reads "Set farm location in Settings" and crates save without it. Nothing else is affected.

Step 2 - Install Python.

1. Go to `python.org` in a browser and download the latest Python 3 installer for Windows (64-bit).
2. Run the installer. **On the very first screen, tick the checkbox at the bottom that says "Add python.exe to PATH"** before clicking "Install Now" - this step is easy to miss and, if skipped, every command below will fail with `'python' is not recognized`.
3. Once installation finishes, click "Close".

Step 3 - Open Command Prompt in the `backend` folder.

1. Open the `C:\Boord\backend` folder in File Explorer.
2. Click into the empty area of the address bar at the top of the window, type `cmd`, and press Enter. A black Command Prompt window opens already pointed at that folder (confirm the prompt reads `C:\Boord\backend>`).

Step 4 - Create a virtual environment. A virtual environment keeps this app's Python packages separate from anything else on the PC. In the Command Prompt window, run:

```
python -m venv .venv
```

This creates a `.venv` folder inside `backend` and takes a few seconds.

Step 5 - Activate the virtual environment.

```
.venv\Scripts\activate
```

The prompt changes to start with `(.venv)` once this has worked - check for that before continuing. You'll need to repeat this activation step every time you open a new Command Prompt window to work with the app (but *not* every time the server itself runs day-to-day - see Step 10).

Step 6 - Install the app's dependencies.

```
pip install -r requirements.txt
```

This downloads everything the app needs (FastAPI, the database library, etc.) into the virtual environment. It can take a few minutes on the first run, depending on the internet connection. If it fails partway with a build/compiler error, the most common cause is a 32-bit Python install - uninstall it and reinstall the 64-bit version from Step 2.

Step 7 - Run the server for the first time.

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

- `--host 0.0.0.0` makes the server reachable from other devices on the network, not just this PC - this is required for field/pack house devices to connect.
- `8000` is just an example port; any free port works, but stick with one number once devices are configured against it.
- The window will print a few lines ending in something like `Uvicorn running on http://0.0.0.0:8000` and then go quiet - that's normal; it means the server is up and waiting. **Leave this Command Prompt window open** - closing it stops the server.

Step 8 - Confirm it's working. On the same PC, open a browser and go to `http://localhost:8000/`. You should see the app's device setup screen (see chapter 3). If you see this, the server itself is working correctly.

Step 9 - Find this PC's network address. Other devices don't use `localhost` - they need this PC's actual address on the network. Open a **second** Command Prompt window (leave the server running in the first one) and run:

```
ipconfig
```

Look for **"IPv4 Address"** under the network adapter that's actually connected (Wi-Fi or Ethernet) - it looks like `192.168.1.50`. Other devices will reach the app at `http://192.168.1.50:8000/` (using this PC's own address and the port from Step 7).

Step 10 - Allow the app through Windows Firewall. Windows will usually pop up a "Windows Defender Firewall has blocked some features of this app" prompt the first time the server starts - tick both "Private networks" and "Public networks" (if shown) and click "Allow access". If that prompt was missed or dismissed, add the rule manually:

1. Open **Windows Security** → **Firewall & network protection** → **Advanced settings**.
2. Click **Inbound Rules** → **New Rule…**.
3. Choose **Port** → Next.
4. Choose **TCP**, and under "Specific local ports" enter the port from Step 7 (e.g. `8000`) → Next.
5. Choose **Allow the connection** → Next.
6. Leave Domain/Private/Public all ticked → Next.
7. Give it a name, e.g. "Boord Server" → Finish.

Step 11 - Stop the PC from going to sleep. The server only works while the PC is awake. Go to **Settings** → **System** → **Power & battery** → **Screen and sleep**, and set "When plugged in, put my device to sleep" to **Never**. (The screen itself can still turn off - that doesn't affect the server - only "sleep"/"hibernate" does.)

> **If the server is a laptop, not a desktop PC:** the above isn't enough by itself - closing the lid puts a laptop to sleep (or shuts it down) by default, regardless of the sleep-timeout setting, which would take the server offline. Search the Start menu for **"Choose what closing the lid does"** and set **"When I close the lid" (on battery and plugged in)** to **"Do nothing"**. Combined with Step 11 above, the server then keeps running lid-closed and plugged in.

Step 12 - (Recommended) Make the server start automatically. Without this step, someone has to manually repeat Steps 3, 5, and 7 every time the PC restarts (e.g. after a power cut or Windows update). To avoid that, first create a new text file at `C:\Boord\start_server.bat` containing exactly:

```
@echo off
cd /d "C:\Boord\backend"
call .venv\Scripts\activate.bat
uvicorn main:app --host 0.0.0.0 --port 8000
```

Then set Windows to run it automatically on startup:

1. First close any server already running: Task Manager (Ctrl+Shift+Esc) → End task on every `python.exe`. From here on the scheduled task owns the server - starting `start_server.bat` by hand as well is what leaves two instances running on different ports (see chapter 12).
2. Open **Task Scheduler** (search for it in the Start menu).
3. Click **Create Basic Task…** and name it exactly **Boord Server**, Next.

> The name is not cosmetic. `update_server.bat` restarts the server by looking this task up by that literal string; under any other name it silently skips the restart and leaves the old backend running while the newly-pulled frontend is served straight from disk - so the version badge updates and the fix appears not to have deployed.

1. Trigger: choose **"When the computer starts"**, Next.
2. Action: choose **"Start a program"**, Next, then browse to and select `C:\Boord\start_server.bat`, Next, Finish.
3. Find the new task in the Task Scheduler Library, right-click → **Properties**, and on the **Settings** tab tick **"If the task fails, restart every:"**, set 1 minute, so a crash recovers itself.
4. Still in Properties, set who it runs as - **and match the trigger to that choice**, or the task never fires:

Account has	General tab	Triggers tab
A Windows password	**"Run whether user is logged on or not"** (asks for the password on save)	leave as **At startup**

Account has	General tab	Triggers tab
No password	**"Run only when user is logged on"**	change to **At log on**

> "At startup" fires before anyone has signed in. Combined with "run only when user is logged on" that condition is never true when it fires, so the task quietly never runs and the server stays down after a reboot - with nothing in the task's History to suggest it was supposed to. A PC with no Windows password auto-signs-in at boot, so "At log on" gets there moments later and is the pairing to use.

Adding a Windows password purely to enable the first row is usually the wrong trade for a server left in an office: it makes the PC stop at a lock screen after a power cut, which is worse for unattended restarts than what it fixes.

1. Right-click the task → **Run**, and check `http://localhost:8000/admin/` loads on the server PC itself.
2. **Then restart the PC and, without touching anything, check the app again from another device.** An auto-start that was never tested this way is worse than none, because everyone assumes it's covered.

With this in place the server behaves like any other piece of office equipment - it just needs the PC left on.

Setting up on Mac or Linux

From the `backend/` folder:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Then, to run it:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

- `--host 0.0.0.0` makes the server reachable from other devices on the network, not just the machine it's running on.
- Find this machine's network address with `ifconfig` (look for `inet` under the active Wi-Fi/Ethernet adapter) so other devices can reach `http://<that-address>:8000/`.
- To keep the server running after closing the terminal, and to have it restart automatically after a reboot, use your platform's standard approach for long-running services (e.g. a `launchd`

agent on Mac, or a `systemd` service on Linux) pointed at the same `uvicorn` command above.

Connecting external users with Tailscale (only if needed)

Skip this section entirely if every device (field, pack house, admin) stays on the farm's own Wi-Fi/LAN - it's only needed if someone **outside** that network has to reach the server, e.g. a remote bookkeeper checking Reports, or a partner farm's own admin accessing their supplier data from elsewhere. Tailscale is a free, private network that lets specific outside devices reach the server securely over the internet, without opening any ports on the farm's router or exposing the app to the public internet.

> **An off-site owner who only wants to check progress** has a separate app of their own - ask whoever set it up for its link. This section is for someone who needs the *full* Admin screen remotely.

Step 1 - Install Tailscale on the server.

1. On the same PC the server runs on, go to `tailscale.com/download` in a browser and download the installer for its operating system (Windows, Mac, or Linux).
2. Run the installer, then sign in when prompted (with a Google, Microsoft, or email account) - this creates your farm's own private Tailscale network (a "tailnet"), separate from anyone else's.
3. Once signed in, Tailscale assigns this server a private address like `100.x.x.x`. Find it by clicking the Tailscale icon in the system tray (Windows) or menu bar (Mac) and reading the address shown there, or by running `tailscale ip -4` in a Command Prompt/terminal.
4. Make sure Tailscale is set to start automatically: right-click its tray icon → **Preferences/Settings** → confirm **"Start on login"** (or equivalent) is ticked, so it reconnects on its own after every restart, the same way the server itself does (Step 12 on Windows).

> **This farm's setup:** the server is signed into a specific Tailscale account, with its own tailnet address. Account, address, and password are kept in a password manager, not this file.

Step 2 - Share (not invite) access to just this one machine.

Tailscale's **"Share"** feature grants an outside person access to only this one server, without adding them to the rest of the farm's tailnet or letting them see any other device on it - this is the right option for an external user, as opposed to "Invite" (which adds someone as a full member of the tailnet).

1. In a browser, go to `login.tailscale.com/admin/machines` and sign in with the same account used in Step 1.
2. Find this server in the machine list, click the **"…"** menu next to it, and choose **Share…**.

3. Enter the external person's email address and send the invite (or copy the generated link and send it yourself, e.g. via WhatsApp or email).
4. The external person accepts the invite, creating their own free Tailscale account if they don't already have one, and installs Tailscale on their own phone, tablet, or PC (same download step as Step 1, but signing in with their own account).

Step 3 - Connecting. Once accepted, the external device can reach this one server at its Tailscale address from anywhere with an internet connection - e.g. `http://100.x.x.x:8000/` - exactly the same way a device on the farm's own Wi-Fi reaches it by LAN address. The device setup screen and roles work identically.

One thing is **not** identical, and it is the reason to read this chapter even if nobody outside the farm needs access: the **Admin app is served only to this server's own console and to the tailnet**. A device on the farm Wi-Fi gets the Field and Pack House screens and nothing else. So a tailnet address is not merely a convenience for working off-site - it is the only way to open Admin from anywhere but the server PC itself.

Step 4 - Removing access later. When an external person no longer needs access (e.g. a contractor's season is over), go back to `login.tailscale.com/admin/machines`, find their device under the server's sharing settings, and remove it - this revokes their access immediately without affecting anyone else.

Enabling the QR camera scanner (HTTPS via Tailscale - required for Field devices)

Android's and iOS's browsers only allow a page to use the camera if it's loaded over HTTPS or from `localhost` - a plain LAN address like `http://192.168.1.50:8000/` fails that check, so **Scan Worker QR** in the Field app (see chapter 4) cannot start the camera on any device that reaches the server that way. Since worker identification is QR-only with no manual picker fallback (see Identifying the worker), this isn't optional - it must be set up before Field devices can capture harvest data at all.

A device on a plain-http address says **"Scanning needs the secure address - check Tailscale is connected, then tell your supervisor"** rather than opening a scanner that cannot work. If a picker reports that, this section is the fix: the phone is opening the app from the LAN address instead of the Tailscale one.

Field devices also need to work over **mobile data** while out picking (they're only back on the farm's own Wi-Fi at the end of the day), which rules out a certificate tied to the server's local network address - that would only ever work from on-site. Tailscale HTTPS is the one setup that covers both requirements at once, since it's reachable over the internet from anywhere the device has a signal, not just the farm's LAN.

Fixing this means putting Tailscale's HTTPS in front of the app, which requires no changes to the app itself:

1. Install Tailscale on the server first (see Connecting external users with Tailscale) if not already done.

2. Turn on certificates for the tailnet: go to `login.tailscale.com/admin/dns` and enable **"HTTPS Certificates"** (requires MagicDNS, which is on by default).
3. On the server, run: `bash tailscale serve --bg --https=443 http://localhost:8000` (use whichever port the app actually runs on). This runs in the background and proxies HTTPS traffic straight through to the existing plain-HTTP app - `uvicorn` / `main.py` don't need to change.
4. Find the exact address to use with `tailscale status` - it looks like `https://<server-name>.<tailnet-name>.ts.net/`.

Once this is done, every Field/Pack House device needs the Tailscale app installed and connected, and pointed at the new address - see Connecting via Tailscale HTTPS in chapter 3.

Remote desktop access via AnyDesk

Separate from Tailscale (which only gets a device to the app itself), AnyDesk is installed on the server PC for full remote-desktop control - useful for admin tasks that need the actual Windows desktop (restarting the Scheduled Task, running `update_server.bat`, Windows updates, etc.) without physically being at the farm office.

> **This farm's server:** has a fixed AnyDesk access code for unattended access. Code and password are kept in a password manager, not this file.

To connect: install AnyDesk on the device you're connecting from, enter this server's access code, and provide the password when prompted.

Uptime alerting (email if the server goes down)

Gets you an email if the server is unreachable for more than an hour. This can't be done by anything running only on the server PC itself - if that PC loses power or its internet entirely, nothing on it can send you anything. Instead it uses a **"dead man's switch"**: the server pings an outside monitoring service every 10 minutes to say "still alive," and that outside service is the one that notices when the pings stop and emails you - it's watching for silence, not waiting to be told about a problem.

Step 1 - Create a healthchecks.io check.

1. Sign up free at healthchecks.io.
2. Create a check (e.g. named "Boord Server").
3. Click **Edit** and set **Period** to **10 minutes** and **Grace Time** to **1 hour** - this means an isolated missed ping (a brief network blip) is tolerated, but if pings stop entirely for over an hour, it emails the account's address.
4. Copy the ping URL shown on the check's page (starts with `https://hc-ping.com/...`).

Step 2 - Save the ping URL on the server. Create a new text file at `heartbeat_url.txt`, in the same folder as `heartbeat.ps1` (the top of the project folder), containing just that one URL and

nothing else. This file is deliberately **not** committed to git - like a password, it's account-specific and shouldn't live in version control (`.gitignore` already excludes it).

Step 3 - Register the heartbeat task. Double-click **`setup_heartbeat.bat`**. It registers a Scheduled Task ("Boord Heartbeat") that runs `heartbeat.ps1` every 10 minutes, which only pings healthchecks.io when `http://localhost:8000/` actually responds - so a crashed/hung server (not just a powered-off PC) also triggers the alert, since the heartbeat is contingent on the app itself working, not just the PC being on.

That's it - as long as the check on healthchecks.io keeps receiving pings, nothing happens. If the server goes down and stays down past the 1-hour grace period, healthchecks.io emails the account used to sign up.

What the ping carries. Each successful ping sends a short line of text with it, visible in the check's log on healthchecks.io. It is there so the release a site is running can be read from the monitoring account instead of phoning someone and asking them to read a screen. It contains exactly:

- the release tag and whether it is a release, a checkout between releases, or unknown
- the version the app screens display
- the database schema revision the code expects, and the one the database is actually at - so a farm that never ran its migrations is visible
- the date of the last backup, and how many are kept

It contains nothing about any person - no worker names, ID numbers, banking details, photographs or counts, no supplier or block names, no crate or lot figures, no coordinates, no file paths and no usernames. The pack house's own name is deliberately left out too: the check already carries whatever name you gave it in your own account.

> **healthchecks.io is a third party.** Anyone with access to that account can read these lines. The ping URL in `heartbeat_url.txt` is a password in every practical sense - anyone holding it can post to your check.

Keeping the server running

However it's started, leave it running continuously during harvest season - the app expects to be a long-lived local service, not something started and stopped around each use. (The automatic nightly backup, chapter 11, only fires if the server happens to be running at 02:00 - see that chapter's note on this limitation. Separately, a night on which nothing changed is skipped on purpose and adds no backup - that's normal, not a fault.)

What happens on first startup

The very first time the server runs, it automatically creates the database and seeds a clean starting baseline:

- Two teams: **Span A** and **Span B** (indunas left blank - fill in via Master Data)
- **8 devices**: `device-01` through `device-05` (field), `device-06` and `device-07` (pack house), and `admin-pc` (admin) - see chapter 3 for how these get assigned to physical devices
- One supplier row representing the pack house's own fruit ("Own fruit") - rename it under Master Data **No admin account, because there are none.** Boord has one admin and no sign-in. Nothing is seeded, there is no password to write down or lose, and nothing to reset. Access is decided by the network a request arrives on - see step 6 of the installer.

No wage rate. Nothing is seeded, and **Calculate** under Payments refuses with "No wage rate has been set" until you enter one under Settings. A wrong block list is obvious the moment somebody opens the Field app; a wrong wage rate produces a perfectly ordinary-looking payslip at the wrong number, and the person most likely to notice is the one being underpaid. So the app will not guess, and it will not quietly calculate everyone at R0.00 either.

No blocks. A new database has an empty block register, and the Field app's Block dropdown reads "(no blocks set up yet - add them in Admin)" until you fill it in. That is deliberate: a block register is the shape of one particular orchard - its labels, varieties, tree counts and hectares - and no two farms share one.

Import yours via **Admin → Settings → Master Data → Blocks → Import**, using `templates/blocks.csv` in the project folder for the column headings (`id` , `name` , `variety` , `trees` , `hectares` , `supplier_id` , `active`). `templates/README.md` describes each column. Both `.csv` and `.xlsx` are accepted, and importing again updates blocks with a matching `id` rather than duplicating them, so it is safe to correct a mistake and re-import.

The setup wizard

You do not have to hunt for any of the above. The first time you open the Admin app on a new install, Boord shows an eight-step setup wizard instead of the usual tabs:

1. **Whose pack house is this** — pack house name, location description, Pack House Code (PHC), how your own fruit is labelled as a supplier (with its PUC and GlobalG.A.P. Number), and the season start date
2. **Where is it** — GPS latitude and longitude, with a map to click on
3. **What you pay per kilogram** — the wage rate
4. **How long fruit may sit in transit** — the two urgency thresholds (optional; the defaults suit most operations)
5. **Your orchard blocks** — download `blocks.csv` , fill it in, load it back
6. **Your people** — the worker import (optional)

7. **Name your stations** — rename the eight device slots (optional)
8. **Finish** — the address to open on your tablets, and anything still outstanding

Steps 1 to 3 are required, because nothing sensible can be guessed for them. The rest can be skipped and done later; the last screen lists anything still outstanding rather than letting you discover it in use.

It is safe to close half way through. Reopening the Admin app returns you to the wizard at the first step you have not finished. It stops appearing once you press **Open Boord** on the last screen, and everything it collects can be changed afterwards under **Settings** — nothing on that screen is decided permanently.

An existing farm upgrading to this version never sees the wizard: Boord checks that the database has no farm name *and* no captured crates before offering it, so a server that has been running is left alone.

⚠️ Do not run `seed_demo.py` on a real farm database

The repo includes `backend/seed_demo.py`, which fills the database with **invented harvest data on past dates** for demoing and testing the app. It is **not** part of normal startup and must never be run against a real farm's database - doing so will inject invented workers, lots, and payments alongside real data with no easy way to tell them apart afterward. It's safe to use only against a throwaway/ test copy of the app.

---

3. Device Setup

The very first time any device (phone, tablet, or computer) opens the app's URL, it shows a one-time **Device Setup** screen instead of jumping straight into a role's screen.

1. Pick this device's **Device ID** from the dropdown. The list comes from the server and shows every active device with its station - e.g. `device-08 — Field Station 6` - so a device added in Master Data → Devices is available here straight away, with no change to the app needed.
2. Tap **Continue**.
3. The device looks up that ID's role and **automatically routes itself** to the right screen - Field, Pack House, or Admin - based on what's configured for that device.
4. The device **remembers its ID** after this (stored in the browser, not on the server) - it won't ask again on future visits, and will jump straight to its role's screen.

> If the setup screen can't reach the server it can't show the list, and falls back to a box for typing the ID by hand. Setup itself still needs the server, since the ID has to be checked against Master Data before the device can be used.

Installing as an app icon (optional, recommended)

Each of the three screens - Field, Pack House, Admin - is a small installable app (a "PWA") with its own name and icon, so a device can show "Veld", "Pakhuis", or "Admin" on its home screen like any other app, instead of a browser tab/bookmark. This isn't required (the browser bookmark/URL works fine on its own), but it makes the right screen one tap away and avoids anyone confusing a browser address bar with the actual app.

Upgrading from an older install. If the device previously had the app under its old name, remove that icon from the home screen and clear the site data for the server address before re-adding it. A leftover icon keeps pointing at the old cached copy, so the device can look like it did not pick up the update.

Important: install from the role's own screen, not the setup screen. Let the device auto-route itself first (steps above), then install from the resulting URL (e.g. `.../field/`, `.../packhouse/`, `.../admin/`) - installing from the very first device-ID entry screen would just save a generic icon, not the role-specific one.

- **Android (Chrome)**: tap the three-dot menu (top right) → **"Add to Home screen"** (or look for an automatic "Install app" banner/icon in the address bar).
- **iPhone/iPad (Safari)**: tap the **Share** icon → **"Add to Home Screen"**.
- **Windows/Mac (Chrome or Edge)**: look for an **install icon** (a monitor with a down-arrow) at the right end of the address bar, or use the three-dot/three-line menu → **"Install [app name]..."**.

Each installs independently with its own icon and name - installing the Field app on a phone doesn't affect what a Pack House tablet or the office admin computer shows.

Connecting via Tailscale HTTPS

Assumes the server-side setup is already done - see Enabling the QR camera scanner in chapter 2. Every Field/Pack House device needs this, not just ones that leave the farm - they need to keep working over mobile data while out picking, which only Tailscale (not a LAN-only setup) can do.

Step 1 - Install and connect Tailscale on this device, signed into the same tailnet as the server (see the next section for keeping it connected reliably on Android).

Step 2 - Point the device at the new address. Open the `https://...ts.net/` address instead of the old LAN IP. This is a different origin, so it'll show the Device Setup screen again (above) - re-enter that device's ID once. Then reinstall the home-screen app icon from the new HTTPS address (see Installing as an app icon above), and delete the old icon that points at the `http://` address.

Keeping Tailscale always-on on Android field devices

Since every Field/Pack House device depends on Tailscale to reach the server (see above), Android's aggressive battery management can silently kill the Tailscale connection in the background, which then breaks the app until someone notices and reopens Tailscale manually. To stop that happening:

1. **Turn on Always-on VPN.** Settings → **Network & Internet** → **VPN** → tap the gear icon next to Tailscale → enable **"Always-on VPN"**. This makes Android keep it running and reconnect it automatically after a restart.
2. **Exempt Tailscale from battery optimization.** Settings → **Apps** → **Tailscale** → **Battery** → set to **"Unrestricted"** (some phones label this "Unmonitored app" or "Don't optimize"). Without this, Android periodically freezes the app in the background and the connection drops until someone opens it again.
3. **Check for a manufacturer-specific app killer.** Samsung, Xiaomi, Huawei, and OnePlus phones ship an extra battery manager on top of stock Android that can re-kill apps even after Step 2 - look for a separate "Sleeping apps," "Protected apps," or "Autostart manager" list under that phone's battery/device-care settings and make sure Tailscale is excluded/allowed there. dontkillmyapp.com has exact steps per phone model.
4. **Stay signed in.** If Tailscale ever gets signed out, it stops connecting entirely until someone signs back in - it won't silently reconnect on its own.

Confirming devices picked up an update (version numbers)

Every screen - Field, Pack House, Admin - shows a small **v{number}** in its top-right corner (e.g. `v1.1`). This is the one reliable way to confirm a device is actually running the latest code after an update and restart, since the Field/Pack House/Admin apps are installable PWAs with an offline

cache (see Installing as an app icon above) - a device can stay on an older cached version even after the server itself has been updated, until its cache is refreshed.

After deploying an update: check the version number on a few devices. If one is behind, do a normal open-close-reopen of the app icon (see chapter 12 - same fix as a stuck "Camera unavailable" screen); a full close and reopen is what lets the app notice and install the new cached version in the background, then show it on the next open.

What the server itself is running is a separate question, and the header cannot answer it - the header says what *that browser* loaded, which is exactly what goes stale. Settings → Server in the admin app asks the server directly and shows both, so "the update did not apply" and "this one device is showing an old cached copy" stop looking the same.

The version number only changes when the code that ships it changes - it's incremented deliberately each time a real update goes out, not tied to the date or any automatic counter.

"Unknown device id"

If the ID isn't recognized, the device shows an error and refuses to continue. This is mainly reachable by typing an ID by hand (the dropdown only ever offers IDs that exist), or if the device was deleted from Master Data after being set up. **Devices are never auto-registered** - an admin must create the device first, in Master Data → Devices, before it can be used. This is intentional: it stops a stray or mistyped device ID from silently attaching itself to the wrong team or role.

Reassigning a device

To point a device at a different role/station (e.g. repurposing a spare tablet), either:

- Clear the device's browser data/cache so it forgets its saved ID and shows the setup screen again, or
- Simply change that device ID's role/station in Master Data → Devices - the physical device keeps using the same ID, but the server now treats it differently on its next check-in.

---

4. Field App - Capturing the Harvest

The field app is what a picking team's device shows once set up. Its job is simple: log every crate as it's weighed, and send a "picking slip" (a load of crates) to the pack house when a truck is ready to take it.

Station header

The top of the screen shows which station/team/induna this device is registered as, and a sync status pill. The pill is **colour-coded** so it can be read at arm's length without stopping to look properly:

Pill	Colour	Meaning
Online - synced	Green	Everything captured on this device has reached the server.
Syncing 3...	Amber	Crates are being uploaded right now.
Offline	Red	The server can't be reached. Capture carries on regardless.
Offline - 3 pending	Red	Same, and 3 captured crates are waiting to be sent.
Online - sync failed, retrying	Amber	The server answered but rejected the last attempt. Nothing is lost; it keeps retrying.

"Offline" covers both having no signal at all and having signal but no route to the farm server - from the device's point of view they're the same thing, and it's the second one that's more common in the orchard.

The offline banner

Whenever a screen can't reach the server it also shows an amber bar under the header - on the field app, "*Offline - crates save to this device and sync when back in range*". It clears itself the moment the connection comes back. Every screen has one (see chapter 6 and chapter 7); the wording differs to suit what that screen does while offline.

Pull down to refresh

On any screen, dragging down from the top of the page reloads that screen's data - the same as the Refresh button where there is one. It's there because an installed app has no browser reload button. Dragging inside a dialog or a list that scrolls on its own just scrolls it as normal.

Identifying the worker: QR scan only

Tap **Scan Worker QR** and point the device's camera at the worker's printed ID badge (see chapter 5 for how badges are made). The app matches the scanned code against known workers and shows the worker's name once matched.

> **Why QR-only, no typing/dropdown?** Picking teams can move dozens of workers through a station in a day - manually finding the right name in a long dropdown list, especially with common surnames, risks misattributing a crate (and therefore wages) to the wrong person. A QR scan is unambiguous.

If the scanned code doesn't match any known worker, the app tells you so and nothing is selected - print or reprint that worker's badge instead of guessing.

Selecting the block

Choose the block being picked from the **Block** dropdown.

Capturing a crate's weight

Use the on-screen numeric keypad to enter the crate's weight in kg, then tap **Save Crate**. The crate is saved immediately to the device - see "Offline-first capture" below - and the running totals update:

- **Crate count** and **total weight** for the current, not-yet-dispatched load
- **Elapsed time**, color-coded green/yellow/red the same way described in chapter 1, so the team can see for themselves when a load has been sitting long enough that it should go.

Offline-first capture

Every crate is written to the device's local storage the instant you tap Save Crate - **capture never waits on a network connection**. In the background, the app checks for a connection every few seconds and quietly syncs anything not yet sent to the server. You do not need to do anything to make this happen, and a signal dropout mid-picking has no effect on capture.

Sending the load: "Send Picking Slip"

When a truck is ready to take the crates picked so far, tap **Send Picking Slip**. You'll be asked for:

- **Crates going now** - defaults to every crate captured so far, but can be reduced if the truck can't take everything (see "Splitting a load" below)
- **Driver name** - required

Once sent, the load moves into "in transit" and will appear in the pack house's queue (see chapter 6). The screen shows a confirmation banner ("Picking Slip Sent - X crates - Y kg dispatched"), and the load also appears under **Dispatched Lots Today** on the same screen.

Splitting a load

Sometimes a truck arrives before a picking round is finished. If you enter a **Crates going now** value lower than the total captured, the app splits the load: the crates going now are dispatched immediately under the current slip number, and the remaining crates are automatically rolled onto a **new** slip number, ready to combine with whatever gets picked next.

> Splitting requires an active connection (the server needs to be the one deciding which crates go on which slip). If offline, the app shows a hint and asks you to either reconnect or just send everything on this load instead of splitting.

The pack house will see both resulting loads flagged as a **split load** with a link back to the other part, so receiving staff know part of this same picking session may arrive separately (see chapter 6).

---

5. Worker ID Badges

Badges are what a field device scans to identify a worker (see chapter 4). An admin generates and prints them from **Master Data** → **Workers**:

- **Print Badges (filtered)** - only the workers currently matching the Farm/Supplier filter
- **Print Badges (selected)** - only the workers with their row checkbox ticked
- **Print Badges (all)** - every active worker

Each badge shows the pack house name, the worker's photo (if one's been captured - see chapter 8), their name, their employee number in large text, and a QR code encoding that employee number.

Badges print **58 mm wide × 78 mm high**, portrait - 1 mm inside the ID card holders on every edge so the printed badge drops into the sleeve without catching. Nine to an A4 sheet, with dashed lines to cut along. Cut on the line and a badge slides straight into a holder; no trimming, and nothing to laminate unless you want to.

Most of the card is the QR code, at **48 mm square**. That's deliberate: it's the part the badge gets used for every day, out in the sun at arm's length, and a bigger code is quicker to catch and can be scanned from further back. It's printed as line art rather than as a picture, so it comes out as sharp as the printer can manage, and it carries the highest error-correction level the format allows - roughly a third of the code can be scratched, smudged or torn and it will still scan.

> ⚠️ **Set the print dialog's scale to 100%.** Browsers default to "Fit to page" or "Shrink to fit" in some print dialogs, which quietly resizes everything - the badges then come out a few millimetres small and won't sit properly in the holder. The print page reminds you of this above the Print button. If a batch comes out the wrong size, this is almost always why.

---

6. Pack House Receiving

This is the screen at the gate where incoming loads are checked in.

The in-transit queue

Every load currently on its way (dispatched from a field station, or logged as an external delivery - see below) appears here, **oldest first**, colored green/yellow/red by how long it's been in transit (the same thresholds as chapter 1, configurable in Settings). Each card shows the slip number, farm/supplier, team, driver, crate count, and total kg.

If a load is part of a **split** (see chapter 4), its card shows a **"Split load - N related slip(s)"** flag. Opening the load shows the related slip(s) with their own status (still in transit, already received, or still being picked) - so receiving staff know to expect (or ask about) the rest of that picking session.

If the server can't be reached

The queue is kept on the device, so the screen still shows **the last queue it loaded** rather than going blank - staff can see what was on its way. The amber offline bar appears, and the timestamp at the top right reads "*offline - last update 4 min ago*" so it's clear the list is a snapshot and not live. It refreshes by itself once the connection is back.

Receiving a load **does** need the connection: confirming a receipt while offline shows "*Could not confirm - check connection and retry*" and nothing is recorded. Unlike field capture, receipts are not queued for later - check the load in again once the connection returns.

Logging an external delivery

For fruit arriving from another farmer (not via this farm's own field devices), tap **+ Log External Delivery** and fill in:

- **Supplier** - the external farm this fruit belongs to (must already exist in Master Data → Suppliers)
- **Crates**
- **Total Kg**
- **Driver**
- **Notes**

This drops the load straight into the in-transit queue like any other load, ready to be received the same way.

Receiving a load

Tapping a card opens the **Receive** modal, with fields in this order:

1. **Expected crates** - read-only, what was dispatched
2. **Actual crates received** - enter what actually arrived
3. **Condition** - tick any that apply: **Good**, **Damaged**, **Sunburn**, **Wet**, **Other** (more than one can be ticked; all ticked values are recorded together)
4. **Notes**
5. **Received by** - required; the app remembers the last name entered here and prefills it next time, to save retyping for the same gate staff member

Confirming moves the load's status to **received** and records the exact receiving time - this is what feeds the admin Dashboard/Reports' "Received" figures.

---

7. Admin - Dashboard

The Dashboard is the admin app's landing screen - a farm-wide overview of what's happening right now, all scoped to a shared filter bar at the top.

Filter bar

- **Supplier** - narrow everything below to one supplier, or leave on "All suppliers"
- **Period start / Period end**, plus quick-fill buttons **Today**, **This Week**, **Season**. The season is a recurring start date (a month and a day) set in Settings; **Season** fills the twelve months from that date, and the season is labelled by the year it starts in. Or set custom dates directly. Whichever preset matches the dates currently selected is highlighted, so it's clear at a glance whether "Today" or a custom range is showing.
- Changing any filter - a preset button, a typed date, or the supplier dropdown - refreshes everything below immediately, so there's no separate Refresh button to remember to press (dragging down from the top of the page still refreshes too, same as everywhere else in the app).

If the server can't be reached

The amber offline bar appears and whatever was last loaded stays on screen, rather than the figures resetting to zero and reading as a real day of no harvest. Numbers refresh by themselves once the connection is back.

There is no session to lose, so a dropped connection cannot sign anybody out. If the Admin app is open over Tailscale and Tailscale itself drops, requests start coming from an address the server does not serve Admin to, and it answers "*The Admin app is only reachable from the server itself or over Tailscale*" - reconnect Tailscale and reload.

KPI cards

Teams Active, Workers Active, Blocks Active (all counted as "had activity in the selected period", not a static list), Total Kg, Total Crates, Avg Kg/Lot, Avg Kg/Crate, and a breakdown of Harvesting / In Transit / Received crates and kg.

The five collapsible lists

Each starts collapsed, showing its running total in the header; tap to expand for the detail rows.

1. **Harvesting** - loads still being picked (not yet dispatched), oldest-first, color-coded the same as pack house
2. **In Transit** - dispatched, not yet received, same ordering/coloring

3. **Received** - newest-received first; tap any row to open that load's crates - see Correcting a captured crate below
4. **Workers** - per-worker crates/kg/amount-due/avg-kg-per-crate, sorted by kg picked (highest first), showing each worker's **Farm/Supplier**
5. **Blocks** - per-block crates/kg/avg-kg-per-crate/avg-kg-per-tree/ avg-kg-per-hectare, sorted alphabetically by block name

Correcting a captured crate

Tapping a row in the **Received** list opens that load's individual crates - time, block, worker, weight, deduction, and net kg - each with an **Edit** link. This is the one place to fix a mistake made at capture time (the wrong worker scanned, a mistyped weight) after the fact; there's no equivalent for a load still Harvesting or In Transit.

Editing a crate lets you change its **Worker**, **Weight (kg)**, and **Deduction (kg)** - nothing else about it (block, device, and when it was picked stay exactly as captured). Saving:

- Recalculates that lot's total crates/kg immediately, so the Received list, Dashboard KPIs, exports, and supplier billing all reflect the correction right away.
- Shows a warning if **wages were already calculated** for the affected worker(s) and period - correcting a crate doesn't retroactively update a wage sheet that was already run, so re-run **Calculate Wages** in Payments for that period afterward. If the crate was reassigned to a different worker, both the old and new worker's periods are checked, since the correction changes both their totals.

> A field device is never able to undo a correction made this way - it doesn't re-send a crate it thinks already synced, and even if it retried after a lost connection, the correction always wins over the device's original capture.

---

8. Admin - Master Data

Master Data is a section of the **Settings** tab, with five subtabs for the pack house's reference data.

Workers

Employee number, name, ID number, bank/account, WhatsApp number, which supplier they belong to, a photo (captured via the device's camera right in the edit form, or uploaded), and active/inactive. Supports CSV/xlsx **Export** and **Import** for bulk edits, and the Print Badges buttons.

Teams

ID (e.g. "A"), name (e.g. "Span A"), induna, active.

Blocks

Each block has a name, variety, tree count, hectares, an optional **supplier** (whose orchard the block is - blank means the pack house's own fruit), and active. Supports CSV/xlsx export/import (the file has a `supplier_id` column) for bulk edits rather than one block at a time - useful whenever the actual block layout changes (e.g. a block gets re-divided by variety).

Import always adds new block IDs and updates existing ones from the file, but never removes anything on its own - a block ID that used to exist but isn't in the file just stays as it was. Tick **"Replace all (deactivate blocks not in the file)"** before importing when the file is a complete replacement for the block list (e.g. after re-dividing a block into smaller ones by variety) - any currently-active block whose ID isn't in the file gets deactivated, not deleted, so historical harvest records that reference it are unaffected.

Devices

ID, role (Field / Receiving / Admin), station name, team, and - for a Field device - the **supplier** it picks for (blank means the pack house's own fruit; every picking slip it sends is attributed there), induna, data capturer, active. This is where new devices must be added before they can complete Device Setup.

Suppliers

Every grower whose fruit passes through this pack house, including your own row (marked "(Own fruit)"). Each carries contact details, a **PUC** (Product Unit Code) and a **GlobalG.A.P. Number**, and a packing rate (per kg, or per crate if per-kg is left at 0) used for the **Facility Billing** panel on this same subtab - pick a supplier and date range, tap **Calculate**, and see how much facility-use fee that supplier owes for fruit actually received (not still in transit) in that period.

9. Admin - Payments

Calculates and exports wages for a filtered period.

- Same filter bar convention as the Dashboard (Farm/Supplier + Today/This Week/Season/ custom dates)
- **Calculate Wages** builds the table below, **grouped by farm/supplier** (own farm first, then alphabetically), each group showing a summary row (worker count, total kg, total wages) followed by that group's workers (name, kg, rate, amount due)
- **Export Wage Sheet** downloads the same grouped breakdown as an `.xlsx` file

> This screen only calculates what's **owed** - marking wages as paid, or tracking payment status, is deliberately not handled inside this app; that's managed in whatever external system/ process the farm already uses for actually paying workers.

---

10. Admin - Reports

Downloadable `.xlsx` reports, all sharing the same filter bar as Payments/ Dashboard (Farm/ Supplier + date range):

Report	Contents
Daily Harvest Summary	Crates/kg by block and team for one day. Always uses just the period start date, even if a wider range is picked - the reports grid flags this one "Daily report only" when that happens.
Lot & Receiving Report	Every lot dispatched in the range, with receiving detail once received
Plukstrokies / Picking Notes	One row per dispatched lot - slip number, date/time, block(s), crates sent vs. received, driver, supplier, condition, notes, received by, weather - sorted by date then team, matching the paper picking-slip notes an induna keeps
Span Pluklys / Team Picking List	One row per team per day, matching the paper "Inligting van die Dag" slip - data capturer, induna, worker count and total deductions, plus repeating columns for every block picked that day (name, kg, deductions) and every lot dispatched that day (crates, time, slip number)
Daaglikse Oesdata / Daily Harvest Data	Kg by date (rows) x block (columns) over a range, with per-day totals (kg, workers, avg/worker, crates, avg/crate) and per-block totals (kg, avg/tree, avg/hectare)
Harvesting List	Loads still being picked, matching the Dashboard's Harvesting list
In Transit List	Dispatched, not-yet-received loads
Pakhuis Ontvangstes / Pack House Receivables	Received loads, matching the pack house's paper "Packhouse Receipt Lists" slip - slip number, date/time split out, the receiving block, farm/supplier, team, driver, crates, kg, and rejected (waste) kg
Worker Harvest Report	Per-worker crates/kg/amount-due/avg-kg-per-crate
Lietsjie Lone / Litchi Wages	One row per worker, with the crates that worker harvested broken out per day - one column per date worked, plus a total - so a whole pay period reads off a single row
Block Harvest Report	Per-block crates/kg/avg-kg-per-crate/avg-kg-per-tree

Every generated report is also saved on the server itself, in `data\reports\` (same filename as the download, e.g. `Daily_Harvest_2026-08-05.xlsx`) - not just wherever the browser that downloaded it happened to save it. Regenerating the same report again (same type and date range) overwrites that file rather than piling up duplicates. This folder isn't covered by the automatic nightly backup (see Data Backup below) - back it up the same way you'd back up anything else in `data\`.

11. Admin - Settings

Server

What this server is actually running, read from the server rather than from the page you are looking at:

- **Release** - the signed release tag the server is checked out on. A checkout between releases shows as e.g. `v3.0-4-gb1182b3 (between releases)`, which is normal on a development machine and unexpected on a farm
- **Database schema** - the migration revision the database is at. If it is behind what the code expects, this says so in red: the migrations did not run, and the app should not be used until that is sorted out
- **Last backup** - the date of the most recent one, and how many are kept
- **This device** - the version this browser has loaded

If the last two disagree, this device is showing cached code and the card says so: close the app completely and reopen it (see chapter 3).

If the daily update check is set up (chapter 2), this card is also where **"v3.1 is available"** appears. Nothing installs itself - that stays a double-click of `update_server.bat` on the server PC.

Data Backup

- **Backup Now** - immediately zips the pack house data and worker photos into a downloadable archive, and adds it to the list below (created date, size, a download link per entry). This always creates a backup, whether anything changed or not
- The same backup also runs **automatically every day at 02:00**, keeping only the **14 most recent** backups (older ones are deleted automatically)
- **The nightly run only saves a backup if the farm data actually changed that day.** On a quiet day - no crates, no receiving, no payments, no edits to workers or master data or settings - nothing is written and no new row appears in the list. That is deliberate: it used to write a full copy every single night, including right through the off-season, which is what made the backups take up so much space. If a full **30 days** pass with no changes at all, one backup is taken anyway as a safety net.
- The archive is the whole database. It used to leave the hourly weather history out, because that one table was 42 MB of a 42.4 MB file; the weather left for the Owner app, and with it the special case. A backup is now well under 100 KB and a restore puts everything back.

> **A gap in the backup list is not a problem in itself.** If the newest backup is a few days or a few weeks old, the ordinary explanation is that nothing has changed since then - there is nothing new to save. Press **Backup Now** any time you want a fresh copy regardless.
>
> ⚠️ **The automatic backup only runs if the server happens to be running at 02:00.** If the machine is switched off overnight, that night's backup simply doesn't happen - there's no separate always-on scheduler behind it. Keep the server running continuously during harvest season (see chapter 2).
>
> **Copy backups off the server.** The 14-backup retention only protects against recent mistakes (e.g. accidentally deleting a worker) - it does **not** protect against the server's disk failing, the PC being stolen, or a fire, since all 14 copies live on that same machine. A copy that leaves the machine is the only real safeguard.

Setting up an automatic off-site copy. Create a text file at `data\backup_copy_to.txt` containing one line: the folder every finished backup should be copied to. Lines starting with `#` are ignored, so it is worth writing down what the folder is:

```
# Off-site backup copies. See MANUAL.md chapter 11.
# NOT a synced cloud folder — these files are not encrypted and contain
# every worker's ID number, banking details and photograph.
E:\BoordBackups
```

From then on, each backup is copied there as soon as it is written, and the Data Backup card reports when it last succeeded, or fails loudly in red if it did not. No file means the feature is simply off. Nothing is ever deleted at the destination - the folder is yours, and Boord only ever adds to it. The folder must already exist: a missing one is reported rather than created, because a folder that appears on the wrong drive is worse than an error message.

> ⚠️ **Do not point this at a synced cloud folder - Google Drive, OneDrive, Dropbox, iCloud.** A backup archive contains `boord.db`, which holds **every worker's SA ID number, bank and account number, WhatsApp number and photograph**, plus every photo file. The archives are not encrypted yet, so copying them into a personal cloud account puts the pack house's worker records on someone else's servers, under a personal login, outside the farm's control. The app warns if the destination looks like one of those folders, but it cannot tell for certain - any folder can be added to a sync client's backup set, and the folder's name proves nothing either way.
>
> **Use somewhere the farm physically controls:** a plugged-in USB stick or external drive, a second disk inside the PC, or a shared folder on another machine on the farm's own network. Cloud destinations become reasonable once backup encryption exists; until then they are a way of losing control of worker data quietly.

> This is not hypothetical. Google Drive was found to be syncing an earlier Boord install, live database included - see chapter 2.

The card also shows **how much space `data\backups\` is using**. Most of it is usually not the nightly archives at all but the `pre_migration_*.db` rollback copies, which are full uncompressed databases. Only three are ever kept, and each schema change evicts the oldest - but if no release has changed the schema for a while, old ones sit there. They are safe to delete by hand once a release has been running happily for a few weeks. Nothing in the app deletes them on a timer, on purpose: code that removes rollback copies to save disk space is the code that one day removes the one somebody needed.

Restoring from a backup

There's currently **no restore button in the app** - Settings only lets you create and download backups, not load one back in. Restoring means manually swapping files on the server, and it's a full replacement, not a merge: **everything captured after the backup's timestamp is lost** (crates, payments, worker/master-data edits, anything) once you restore it - there's no way to selectively bring back just part of it.

Before restoring, protect today's data in case you need it back: Copy the *current* `data\boord.db` and `data\photos\` folder somewhere safe first (e.g. rename them or copy them out of `data\`). If the restore turns out to be the wrong call, you'll still have what was there before you overwrote it.

Steps:

1. Get the backup zip you want to restore - either downloaded from Settings → Data Backup, or directly from `data\backups\` on the server (filenames are timestamped, e.g. `backup_20260804_020000.zip`). The `pre_migration_*.db` files in that same folder are **not** these backups and are not restored this way - see Database changes on update.
2. Stop the server (see Stopping, starting, and restarting the server).
3. Unzip the backup - it contains `boord.db` and a `photos\` folder.
4. Copy those into `data\`, replacing the current `data\boord.db` and `data\photos\`.
5. Start the server again. Everything is back - harvest data, workers, payments, settings and photos all come from the archive, and there is nothing to rebuild afterwards.

Pack House settings

Pack House name, location description, **Pack House Code (PHC)**, the season start date (a month and a day that repeats every year - drives what "Season" means throughout the app; the current season year is shown beside it), the green→yellow and yellow→red urgency thresholds in minutes (used everywhere the traffic-light coloring appears - chapter 1), and GPS coordinates (latitude/longitude, or pick a location on the map) - setting these enables automatic weather capture on every dispatched load.

Harvest rate

The per-kg wage rate used by Payments.

Header weather

The admin screen's header shows a live current-weather readout (temperature, condition, humidity) next to the farm name/clock - a quick at-a-glance check of conditions without leaving the app.

12. Troubleshooting / FAQ

The version in the corner and the server disagree Settings → Server shows what the server is running; the header shows what that browser loaded. If they differ, that device is serving cached code: close the app completely (not just backgrounded) and reopen it. If instead Settings → Server says the **database schema** is behind, that is a different and more serious thing - the migrations did not run. Restart the server, and if that does not fix it, stop using the app and see Database changes on update.

Settings → Server says the release is "unknown" The server could not read its own version from git. The card shows why. The usual cause on Windows is git refusing a repository owned by a different user account than the one the server runs as ("detected dubious ownership"). The app works normally; only the version reporting is affected. Running `update_server.bat` once fixes the display either way, by leaving a note of the tag it installed.

"Unknown device id" on a device's first setup The device ID hasn't been created yet. An admin must add it in Master Data → Devices first - see chapter 3.

"Scanning needs the secure address" when tapping Scan Worker QR The device is reaching the server over plain `http://` (a LAN IP) rather than HTTPS - Android/iOS block camera access on any page that isn't a secure origin, regardless of camera permissions. See Enabling the QR camera scanner to set up Tailscale HTTPS, which fixes this for every device at once. If that is already set up, this device is opening the app from the old LAN address - re-add it from the `https://...ts.net/` one and delete the old icon.

"Camera unavailable" when tapping Scan Worker QR Different problem: the address is fine, but the camera itself would not start. Usually camera permission was denied for the browser, or another app is holding the camera. Check the phone's app permissions, close anything else using the camera, and reopen. Tailscale being disconnected can also produce this, since the device then falls back to the LAN address.

A device shows an older version number than expected Its offline app cache hasn't refreshed yet - see Confirming devices picked up an update. Fully close the app (not just background it) and reopen it; if that doesn't clear it, clear the site's data in the browser and reopen.

> Each screen keeps its **own** copy of the app, so updating the field device does nothing for the pack house device and vice versa. After a deploy, every device needs one open while it can still reach the server. Check the version badge on each rather than assuming one device's update covers the rest.

A device newly added in Master Data doesn't appear on the setup screen The setup screen reads the list from the server each time it loads, so a device added in Master Data → Devices should appear as soon as the page is reloaded. If it doesn't: check the device is marked **active** (inactive devices are deliberately not offered), and that the setup screen can actually reach the

server - if it can't, it falls back to a type-in box and shows no list at all. Typing the ID by hand always works as long as the ID exists in Master Data.

"QR code doesn't match a known worker" when scanning a badge Either the worker doesn't exist in Master Data → Workers yet, or their badge is stale/misprinted. Reprint the badge from Workers after confirming the worker record exists (see chapter 5).

Field app shows "Online - sync failed, retrying" The device reached the server but the server rejected the last sync attempt. Nothing is lost - captured crates stay queued on the device and the app keeps retrying automatically every few seconds. (If the server can't be reached at all, the pill reads **Offline** instead, in red.)

A screen says "Offline" but the device clearly has Wi-Fi "Offline" means *this app can't reach the farm server*, which is not the same as having no signal - the far more common case in the orchard is a device still associated with an access point that has no route back to the server PC. Check the server PC is on and awake, and (for anyone off-site) that **Tailscale** shows Connected. Field capture is unaffected either way: crates keep saving to the device and go up automatically when the connection returns.

Every screen says "Offline", including on the server PC itself When even the server PC's own browser shows the amber bar, work down this list in order - each step rules out a whole layer, and skipping to the end wastes the most time:

1. **Is the server actually running - exactly once?** Open Task Manager (Ctrl+Shift+Esc) → Details and look for `python.exe`. A Windows restart (power cut, update) leaves the server down unless the Task Scheduler auto-start in chapter 2 Step 12 was set up - that step is optional, and without it nothing brings the server back by itself. Start it with `start_server.bat`.

Seeing **more than one** `python.exe` group matters just as much as seeing none: a second server started while the first was still running binds a different port, so `tailscale serve` may be forwarding to one instance while you are reading the other, and the two disagree about what has been saved. End every `python.exe` task, confirm none remain, then start exactly one.

1. **Try `http://localhost:8000/admin/` on the server PC.** This bypasses the network entirely. If this works but the `.ts.net` address doesn't, the app and its data are fine and the problem is purely Tailscale - continue to step 3. If this *also* fails, go back to step 1.
2. **Check Tailscale.** Its tray icon should read Connected. If the browser shows `ERR_NAME_NOT_RESOLVED` on the `.ts.net` address, the name isn't resolving at all: those names exist only in Tailscale's own MagicDNS, never in ordinary public DNS, which is why an unrelated site still loads fine while the farm's own address doesn't. Reconnect Tailscale, run `ipconfig /flushdns`, and confirm **MagicDNS** is still enabled at `login.tailscale.com/admin/dns`.

> **A working internet connection tells you nothing here.** The farm server is reached over Tailscale, not the public internet, so an ordinary web page loading normally is not evidence the app should be reachable - the two use different DNS and different routes. Test the app's own address, not a search engine.

> The header weather readout and the conditions stamped onto each crate as it is dispatched both call out to the weather service. If those fail while the rest of the app works, the server is fine and it's that outbound call failing - the header simply shows nothing and a crate saves without its weather, rather than anything breaking.

"Reconnect to send a partial load, or dispatch everything now" when sending a picking slip
You tried to split a load (send fewer crates than captured) while offline. Splitting needs a connection because the server decides the split; either wait for a connection or send the full load instead.

"The Admin app is only reachable from the server itself or over Tailscale" The Admin screens are served to the server's own console and to the tailnet, and nothing else - the Field and Pack House screens are what the farm Wi-Fi gets. Either open Admin on the server PC at `http://localhost:8000/`, or connect Tailscale on the device you are using and open the `https://...ts.net/` address instead of the LAN one. There is no password involved and nothing to reset; the address is the whole of it.

Annexe A: Data Field Reference

Field-by-field reference for every stored record type, with a realistic example value and any limitation worth knowing. Types/defaults/foreign keys below match `backend/models.py` exactly.

Worker

Field	Type	Example	Notes / Limitations
`id`	text	`"001"`	Primary key - the employee number. Must be unique and must match the number printed on the worker's badge.
`first_name`	text	`"Sipho"`	
`last_name`	text	`"Dlamini"`	
`id_number`	text	`"8501015800083"`	Free text, not validated.
`bank`	text	`"Nedbank"`	
`account`	text	`"9963334018"`	
`whatsapp_number`	text	`"+27821234567"`	Optional, not currently used to send anything automatically.
`supplier_id`	number (optional)	`2`	Which farm this worker belongs to. Left blank for the farm's own workers.
`photo_filename`	text	`"001.jpg"`	Set automatically when a photo is captured/uploaded - not hand-edited.
`active`	true/false	`true`	Inactive workers are hidden from most pickers/dropdowns but kept for history.

Team

Field	Type	Example	Notes / Limitations
`id`	text	`"A"`	Primary key. Short code, e.g. a single letter.
`name`	text	`"Span A"`	
`induna`	text	`"Samuel Mthembu"`	
`active`	true/false	`true`	

Block

Field	Type	Example	Notes / Limitations
`id`	text	`"8a"`	Primary key - a real block label the orchard uses. Imported by the pack house; not free-form.
`name`	text	`"Block 8a"`	
`variety`	text	`"Mauritius"`	
`trees`	number	`450`	Whole number of trees on the block.
`hectares`	decimal	`2.3`	
`supplier_id`	number (optional)	`2`	Which supplier's orchard this block is. Blank = the pack house's own fruit.
`active`	true/false	`true`	

Device

Field	Type	Example	Notes / Limitations
`id`	text	`"device-01"`	Primary key - must be entered exactly on the physical device's setup screen (chapter 3).
`role`	enum	`"field"`	Must be exactly one of `field`, `packhouse`, `admin` (shown as Field / Receiving / Admin).
`station`	text	`"Field Station 1"`	
`team_id`	text (optional)	`"A"`	Which team this field device belongs to. Not applicable to Receiving/Admin devices.
`supplier_id`	number (optional)	`2`	For a Field device: which supplier it picks for. Blank = the pack house's own fruit. Picking slips it sends are attributed there.
`induna`	text	`"Samuel Mthembu"`	
`data_capturer`	text	`""`	Free text, optional.
`active`	true/false	`true`	
`last_seen`	timestamp	(auto)	Updated automatically every time the device checks in - not editable.

Supplier

Field	Type	Example	Notes / Limitations
`id`	number	`1`	Primary key, auto-assigned.
`name`	text	`"Jansen Boerdery"`	
`contact_name` / `contact_phone` / `contact_email`	text	`"Piet Jansen"` / `"082-555-1234"` / —	All optional.
`puc`	text	`"PUC1234567"`	Product Unit Code - the grower's registered production unit. Optional.
`global_gap_number`	text	`"4049928123456"`	GlobalG.A.P. Number (GGN). Optional.
`is_own_farm`	true/ false	`false`	Exactly one supplier row in the whole system should ever have this set to `true` - that row represents the pack house's own fruit.
`packing_rate_per_kg`	decimal	`1.50`	If greater than 0, this rate is used for Facility Billing; otherwise the per-crate rate below is used.
`packing_rate_per_crate`	decimal	`25.00`	Only used when the per-kg rate is 0.
`active`	true/ false	`true`	

HarvestRecord (one crate)

Field	Type	Example	Notes / Limitations
`uuid`	text	(auto-generated)	Primary key, generated on the capturing device - lets a retry after a dropped connection safely resubmit the same crate without duplicating it.
`timestamp`	timestamp	`2026-07-08T09:14:00Z`	When the crate was captured.
`worker_id`		`"001"`	

Field	Type	Example	Notes / Limitations
	text (optional)		Set via the QR badge scan (chapter 4) - required in practice even though nullable in the schema.
`block_id`	text (optional)	`"8a"`	Required in practice.
`weight_kg`	decimal	`12.4`	A single crate's weight - typically in the 8-20kg range for litchi crates.
`deduction_kg`	decimal	`0.0`	"Aftrekkings" (waste/reject deduction), subtracted from the crate's weight everywhere a total is calculated. Always `0` at capture time - there's no field-app screen for it - but an admin can set a non-zero value when correcting a crate after it's received.
`lot_id`	number (optional)	`21`	Always set at capture time, pointing at a placeholder load that becomes a real dispatch once "Send Picking Slip" is used.
`weather_temp` / `weather_humidity` / `weather_condition`	decimal / decimal / text	`24.1` / `55` / `"Clear"`	Conditions at the farm the moment the crate synced to the server, only if GPS coordinates are set in Settings - same source as the per-lot weather on Lot below, but stamped once per crate rather than once per dispatch.
`edited_at` / `edited_by`	timestamp / text (both optional)	— / `"admin"`	Set only when an admin corrects this crate after capture - never touched by the field app. Once set, a re-sync of the capturing device's original copy of this crate can no longer overwrite the correction.

Lot (a picking slip / load)

Field	Type	Example	Notes / Limitations
`slip_number`	text	`"device-01-20260708091400"`	Unique. Auto-generated from the device ID and a timestamp; a split creates a second, related slip number.
`timestamp`	timestamp	`2026-07-08T09:14:00Z`	Dispatch time - the basis for the urgency color-coding.
`supplier_id`	number	`1`	Which farm this load's fruit belongs to.
`driver`	text	`"Sello"`	Required when dispatching.
`total_crates` / `total_kg`	number / decimal	`18` / `238.5`	
`status`	enum	`"in_transit"`	One of `created` (still being picked), `in_transit` (dispatched), `received`, `processing_complete`.
`received_at`	timestamp (optional)	—	Set the moment pack house staff confirm receipt.
`weather_temp` / `weather_humidity` / `weather_condition`	decimal / decimal / text	`24.1` / `55` / `"Clear"`	Captured automatically at dispatch time (or when an external delivery is logged), only if GPS coordinates are set in Settings.
`split_from_slip_number`	text (optional)	—	Only set on the "leftover" slip created by a split (chapter 4) - points back at the original slip it was carved out of.

ReceivingRecord

Field	Type	Example	Notes / Limitations
`lot_id`	number	`21`	Which load this receiving entry belongs to.
`expected_crates` / `actual_crates`	number	`18` / `17`	
`discrepancy`	number	`-1`	Calculated automatically as `actual - expected` - not entered directly.
`condition`	text	`"Good, Sunburn"`	Free text, but in practice a comma-joined list of whichever of Good/Damaged/ Sunburn/Wet/Other were ticked.
`received_by`	text	`"Elsa"`	Required - the app remembers and prefills the last value entered on this device.

Payment

Field	Type	Example	Notes / Limitations
`worker_id`	text	`"001"`	
`period_start` / `period_end`	date	`2026-07-01` / `2026-07-31`	
`total_kg`	decimal	`318.6`	Sum of that worker's net crate weights in the period.
`rate_applied`	decimal	`3.00`	The per-kg rate in effect when calculated.
`amount_due`	decimal	`955.80`	Calculated - not directly editable. There is intentionally no "paid" flag or status field on this record (chapter 9); payment status is tracked outside this app.

RateSetting

Field	Type	Example	Notes / Limitations
`effective_date`	date	`2026-07-01`	
`rate_type`	enum	`"per_kg"`	`per_kg` or `per_crate_tier`.
`default_rate_per_kg`	decimal	`3.00`	Used when `rate_type` is `per_kg`.
`tier_rates_json`			

Field	Type	Example	Notes / Limitations
	text (JSON)	`{"1": 2.5, "1.5": 3.5, "2": 4.5}`	Only relevant if using `per_crate_tier` - maps a crate-size tier to its own rate.

SystemSetting

Field	Type	Example	Notes / Limitations
`packhouse_name`	text	`"Boord Pack House"`	Blank on a fresh install - set it in Settings.
`packhouse_location`	text	`"Bekfontein, Mpumalanga"` (example)	Free text description, not used for weather (see `gps_lat` / `gps_lon`).
`packhouse_code`	text	`"PH1234567"`	PHC - the pack house's registered code. Blank until set in Settings.
`season_start_month` / `season_start_day`	number	`9` / `1`	The recurring season start (defaults to `1` / `1` = calendar year). Drives what "Season" means throughout the app (chapters 7, 9, 10).
`current_harvest_year`	number	`2026`	Derived label for the current season - the year it starts in. Kept in sync from the season start date.
`green_to_yellow_minutes` / `yellow_to_red_minutes`	number	`90` / `150`	The urgency color thresholds referenced throughout chapters 1, 4, 6, 7.
`gps_lat` / `gps_lon`	decimal (optional)	`-25.572747` / `31.606722`	Setting both enables automatic weather capture on dispatch.